

CURSO COMPLETO

PYTHON

IMPRESSONADOR

PARTE 2 • BÁSICO

SUMÁRIO

MÓDULO 14	CRIANDO SUAS FUNÇÕES EM PYTHON
81	Functions no Python 4
82	Retornar um valor na Function 6
83	Argumentos e parâmetros em uma Function 7
84	Aplicação em um Exemplo de argumento 8
85	Vários parâmetros e tipos de parâmetros 9
86	Dicas e Exemplos com Argumentos em Function 11
87	Valores padrões para argumentos 13
88	Falando um pouco mais dos returns das functions 15
89	Return com mais de uma informação..... 16
90	Docstrings e annotations 18
91	Exceções e erros em funções 20
92	Múltiplos Argumentos para uma Function 24
93	Ordem dos Tipos de Argumento 26

MÓDULO 15	LIST COMPREHENSION
94	O que é List Comprehension 28
95	Exemplo de List Comprehension 31
96	List Comprehension filtrando com if 32
97	Tratando casos em List Comprehension..... 33
98	List Comprehension não serve apenas para criar listas..... 34

MÓDULO 17	MÓDULOS, BIBLIOTECAS E INTRODUÇÃO À ORIENTAÇÃO A OBJETO
99	O que é Orientação a Objeto e Por que isso importa 36
100	O que são módulos e qual a importância 37
101	Como ler um Módulo 39
102	Módulo time 40
103	Módulo de Gráficos – Exibindo Gráficos no Python 42
104	Mais edições de Gráficos e Módulo Numpy 45
105	Como instalar módulos novos..... 49



PYTHON

MÓDULO 14

CRIANDO SUAS FUNÇÕES EM PYTHON



Módulo 14 – Criando suas funções em Python – Functions no Python (1/2)

```
#Operação rotineira: Cálculo da média
nota1 = input('Insira nota 1:')
nota2 = input('Insira nota 2:')
media = (float(nota1) + float(nota2))/2
print('A média das notas é : {}'.format(media))
```

```
Insira nota 1:7
Insira nota 2:6
A média das notas é : 6.5.
```

```
#Criando uma função calcular_media:
def calcular_media(nota1,nota2):
    nota1 = input('Insira nota 1:')
    nota2 = input('Insira nota 2:')
    media = (float(nota1) + float(nota2))/2
    return print('A média das notas é : {}'.format(media))
```

```
#Chamando a função que acabamos de criar:
calcular_media(10,5)
```

```
Insira nota 1:5
Insira nota 2:6
A média das notas é : 5.5.
```

Função **calcular_media** criada para esse programa que necessita de 2 parâmetros para rodar, nota1 (Ex:10) e nota2 (Ex:5).

Diversas vezes usamos funções nos nossos códigos. O Python já nos fornece dezenas delas!

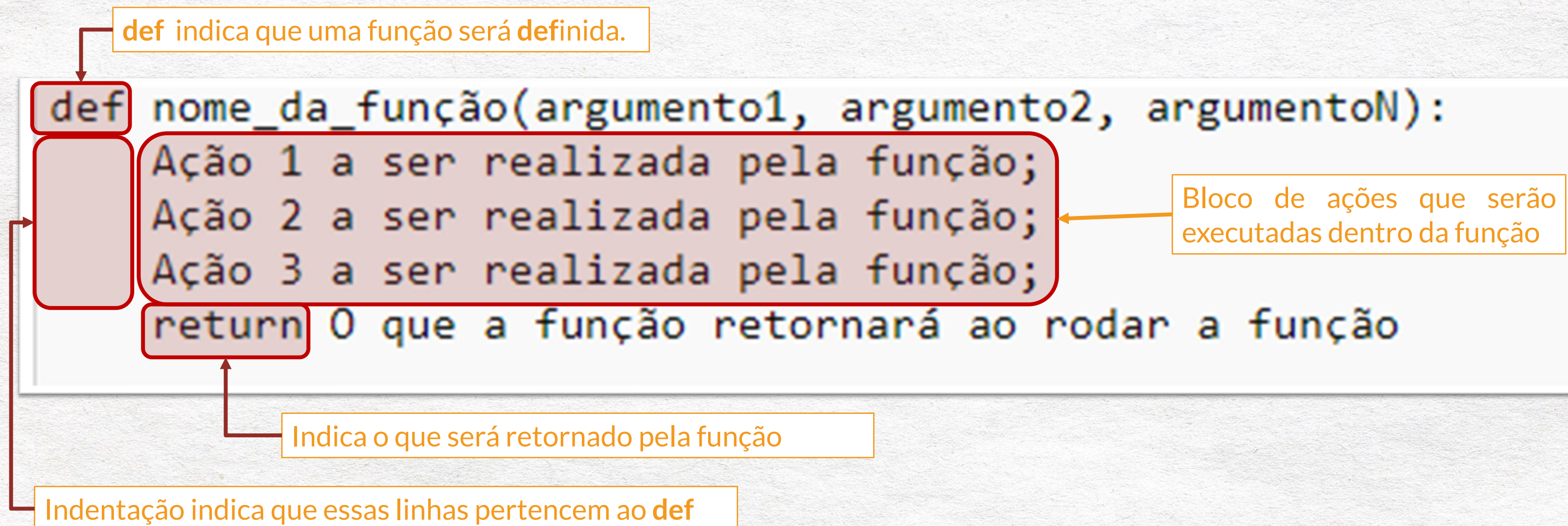
Mas e se eu faço uma operação frequentemente? Será que poderíamos criar uma função que atenda essa atividade?

Veja o exemplo ao lado. Um simples, apenas para entendermos o conceito.

Imagine que sempre precisamos calcular a media de 2 notas. Vamos criar uma função **calcular_media** que nos permita realizar essa operação.

Módulo 14 – Criando suas funções em Python – Functions no Python (2/2)

Vamos entender a estrutura no Python para a criação de uma função.



Módulo 14 – Criando suas funções em Python – Retornar um valor na Function

```
#Operação rotineira: Cálculo da média
nota1 = input('Insira nota 1:')
nota2 = input('Insira nota 2:')
media = (float(nota1) + float(nota2))/2
print('A média das notas é : {}'.format(media))
```

```
Insira nota 1:7
Insira nota 2:6
A média das notas é : 6.5.
```

```
#Criando uma função calcular_media:
def calcular_media(nota1,nota2):
    nota1 = input('Insira nota 1:')
    nota2 = input('Insira nota 2:')
    media = (float(nota1) + float(nota2))/2
    return print('A média das notas é : {}'.format(media))
```

```
#Chamando a função que acabamos de criar:
calcular_media(10,5)
```

```
Insira nota 1:5
Insira nota 2:6
A média das notas é : 5.5.
```

Vamos voltar para nosso exemplo anterior onde calculamos a média de duas notas.

Primeiros vamos entender o que é “Retornar um valor”.

Como estamos falando em functions, estamos falando de criarmos um código que a partir de algum input ela processa os dados e retorna um resultado.

No nosso caso por exemplo:

Inputs -> nota1 e nota2

Resultado retornado -> Média das notas

Se olharmos o código da nossa função **calcular_media**, veremos que ao final temos uma linha **RETURN**.

Essa linha informa o que será retornado nessa função. No nosso exemplo, um print com a média das notas.

Essa resposta geralmente são: Listas, variáveis, dicionários, etc...



Módulo 14 – Criando suas funções em Python – Argumentos e parâmetros em uma Function

```
#Operação rotineira: Cálculo da média
nota1 = input('Insira nota 1:')
nota2 = input('Insira nota 2:')
media = (float(nota1) + float(nota2))/2
print('A média das notas é : {}'.format(media))
```

```
Insira nota 1:7
Insira nota 2:6
A média das notas é : 6.5.
```

```
#Criando uma função calcular media:
```

```
def calcular_media(nota1,nota2):
    nota1 = input('Insira nota 1:')
    nota2 = input('Insira nota 2:')
    media = (float(nota1) + float(nota2))/2
    return print('A média das notas é : {}'.format(media))
```

```
#Chamando a função que acabamos de criar:
calcular_media(10,5)
```

```
Insira nota 1:5
Insira nota 2:6
A média das notas é : 5.5.
```

Como citamos no slide anterior, para que essa função funcione são necessários dois inputs:

- nota1;
- nota2.

Esses inputs são chamados de **argumentos** ou **parâmetros**.

Importante dizer que os argumentos não são obrigatórios.

Os argumentos serão sempre definidos dentro dos parênteses da nossa linha de código que se inicia com **def**.

Módulo 14 – Criando suas funções em Python – Aplicação em um Exemplo de argumento

Agora vamos ver um exemplo onde teremos que criar uma **function** para resolver o desafio. O desafio consiste em analisar uma lista de produtos e enviar instruções dizendo quais produtos devem ser enviados para a área de bebidas alcóolicas.

Cada produto tem um código, por exemplo:

Vinho -> BEB12302

Guaraná -> BSA11104

As bebidas não alcóolicas começam com **BSA**, enquanto as bebidas alcóolicas começam com **BEB**.

```
Enviar beb46275 para setor de bebidas alcóolicas
Enviar BEB21365 para setor de bebidas alcóolicas
Enviar BEB31623 para setor de bebidas alcóolicas
Enviar BEB73344 para setor de bebidas alcóolicas
Enviar BEB80694 para setor de bebidas alcóolicas
Enviar BEB19495 para setor de bebidas alcóolicas
Enviar BEB97471 para setor de bebidas alcóolicas
Enviar BEB62362 para setor de bebidas alcóolicas
Enviar BEB85146 para setor de bebidas alcóolicas
Enviar BEB48898 para setor de bebidas alcóolicas
Enviar BEB79496 para setor de bebidas alcóolicas
Enviar BEB15385 para setor de bebidas alcóolicas
Enviar BEB24213 para setor de bebidas alcóolicas
Enviar BEB56262 para setor de bebidas alcóolicas
Enviar BEB75073 para setor de bebidas alcóolicas
```

```
def ehalcoolico(bebida):
    bebida = bebida.upper()
    if 'BEB' in bebida:
        return True
    else:
        return False
```

Function que retorna **True** se a bebida for alcóolica e **False** se a bebida não for alcóolica

A lista de produtos é percorrida e cada **produto** é enviado como argumento da função **ehalcoolico** e testado se é uma bebida alcóolica

```
produtos = ['beb46275', 'TFA23962', 'TFA64715', 'TFA69555', 'TFA56743', 'BSA45510', 'TF

#percorrer toda a minha Lista de produtos
#pra cada produto, verificar se ele é bebida alcoolica
#se for bebida alcoolica, exibir a mensagem Enviar....

for produto in produtos:
    if ehalcoolico(produto):
        print('Enviar {} para setor de bebidas alcóolicas'.format(produto))
```

Caso seja uma bebida alcóolica, uma mensagem com o código do produto será printada



Módulo 14 – Criando suas funções em Python – Vários parâmetros e tipos de parâmetros (1/2)

Na nossa função `Calcular_media`, já usamos mais de um parâmetro.

Mas vamos entender alguns detalhes de como trabalhar com mais de um parâmetro de uma função.

Existem duas formas distintas de informar parâmetros para funções:

- 1) Em ordem (**positional argument**);
- 2) Com o nome do argumento (**keyword argument**)

Vamos entender essa diferença melhor olhando o exemplo ao lado:

Aqui temos uma função que classifica produtos baseado na sua categoria. Em especial, as bebidas, se são alcóolicas ou não.

```
def eh_da_categoria(bebida, cod_categoria):
    bebida = bebida.upper()
    if cod_categoria in bebida:
        return True
    else:
        return False
```

```
#Usando ordem:
for produto in produtos:
    if eh_da_categoria(produto, 'BEB'):
        print('Enviar {} para setor de bebidas alcóolicas'.format(produto))
    elif eh_da_categoria(produto, 'BSA'):
        print('Enviar {} para setor de bebidas não alcóolicas'.format(produto))

#Usando palavra chave:
for produto in produtos:
    if eh_da_categoria(cod_categoria='BEB', bebida=produto):
        print('Enviar {} para setor de bebidas alcóolicas'.format(produto))
    elif eh_da_categoria(produto, 'BSA'):
        print('Enviar {} para setor de bebidas não alcóolicas'.format(produto))

#Só podemos usar um método, o caso abaixo retornará erro!
for produto in produtos:
    if eh_da_categoria(cod_categoria='BEB', produto):
        print('Enviar {} para setor de bebidas alcóolicas'.format(produto))
    elif eh_da_categoria(produto, 'BSA'):
        print('Enviar {} para setor de bebidas não alcóolicas'.format(produto))
```

Argumentos na mesma ordem

Argumentos por palavra chave

ERRO!



Módulo 14 – Criando suas funções em Python – Vários parâmetros e tipos de parâmetros (2/2)

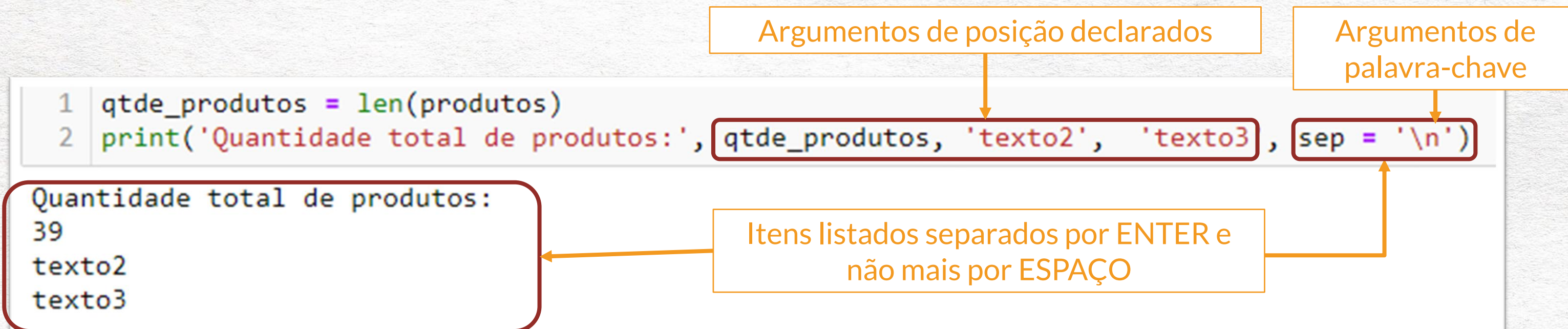
Vamos entender um pouco melhor o erro do slide anterior.

Quando usamos um parâmetro de keyword no início da declaração de parâmetros, devemos seguir usando Keywords para todos os outros argumentos.

Vamos entender um caso onde podemos usar tanto os argumentos de posição quanto de palavras-chaves em uma mesma linha de código. Vamos aproveitar para aprender um novo argumento:

sep =

Este argumento nos ajuda a apresentar os dados separando-os conforme a informação que seja fornecida por nós. Vamos para o exemplo:



Módulo 14 – Criando suas funções em Python – Dicas e Exemplos com Argumentos em Function (1/2)

Nessa aula vamos ver alguns exemplos de funções que já usamos e seus respectivos parâmetros (de palavras-chaves, de posicionamento ou sem parâmetros).

- **upper()** -> não tem parâmetros

```
cod_produto = 'beb12304'  
print(cod_produto.upper())  
  
BEB12304
```

Se tentarmos passar algum valor como parâmetro no **upper()**, ocorrerá um erro.

- **sort()** -> apenas parâmetros keyword

```
vendas_ano = [100, 200, 50, 90, 240, 300, 55, 10, 789, 60]  
vendas_ano.sort(reverse = True)  
print(vendas_ano)  
  
[789, 300, 240, 200, 100, 90, 60, 55, 50, 10]
```

Ordenando uma lista em ordem decrescente através do método **sort()**. Para isso, devemos escrever: **reverse = True**.

O argumento **reverse** é um argumento de palavra-chave.

Módulo 14 – Criando suas funções em Python – Dicas e Exemplos com Argumentos em Function (2/2)

Para saber mais sobre uma determinada função e quais tipos de parâmetros essa função aceita, o recomendado é que você faça uma pesquisa na documentação dessa função.

- `extend(lista)` -> 1 parâmetro obrigatório

```
vendas_ano = [100, 200, 50, 90, 240, 300, 55, 10, 789, 60]
vendas_novdez = [500, 1555]
vendas_ano.extend(vendas_novdez)
print(vendas_ano)
```

```
[100, 200, 50, 90, 240, 300, 55, 10, 789, 60, 500, 1555]
```

O `extend` possui 1 parâmetro de posição que recebe uma lista `vendas_novdez` e adiciona os itens dessa lista dentro da lista `vendas_ano`

- Nossa função `eh_da_categoria(bebida, cod_categoria)` -> 2 parâmetros de posição obrigatórios

```
def eh_da_categoria(bebida, cod_categoria):
    bebida = bebida.upper()
    if cod_categoria in bebida:
        return True
    else:
        return False
```

```
if eh_da_categoria(cod_produto, "BEB"):
    print("É uma bebida alcoólica")
```

```
É uma bebida alcoólica
```

A função `eh_da_categoria` possui 2 parâmetros de posição.

Dica: Quando você não se lembrar de quais valores determinada função precisa para funcionar, no jupyter e com o cursor no nome da função, você deverá pressionar **SHIFT + TAB** para as informações sobre essa função serem mostradas.



Módulo 14 – Criando suas funções em Python – Valores padrões para argumentos (1/2)

Já reparou que em algumas funções existem padrões já estabelecidos no Python?

Não? Vamos para um caso:

Método `.sort()`. Sempre que fazemos este método, o padrão do Python é ordenar os dados de forma crescente como na figura ao lado.

No entanto, podemos forçar o Python a não usar o padrão fornecendo a ele uma palavra-chave:

`sort(reverse = True)`

Assim, a nossa lista será ordenada de forma decrescente.

O que acontece aqui é que o padrão do Python omite sua configuração padrão:

`sort(reverse=False)`

Isso acontece pois a função `sort` foi programada dessa forma!

```
1 vendas = [100,30,70,94,15,65,35]
2 vendas.sort() ← Reverse=False omitido (Padrão)
3 print('Ordem crescente')
4 print(vendas)
5 vendas.sort(reverse=True) ← Keyword
6 print('Ordem decrescente')
7 print(vendas)
```

Ordem crescente

[15, 30, 35, 65, 70, 94, 100]

Ordem decrescente

[100, 94, 70, 65, 35, 30, 15]

Módulo 14 – Criando suas funções em Python – Valores padrões para argumentos (2/2)

O mais interessante, é que podemos fazer exatamente a mesma coisa nas funções criadas por nós!

Vamos para um exemplo que cria códigos de produtos. No entanto, temos duas possibilidades de código:

- Todo em letras maiúsculas (M);
- Todo em letras minúsculas (m).

Nosso padrão será minúsculas, mas o usuário poderá usar a palavra chave `padrao='M'` para alterar para letras maiúsculas.

Perceba que usamos o IF para definir o que acontecerá dependendo do padrão fornecido pelo usuário.

Ao lado apresentamos a função e o resultado dos casos. Perceba que no primeiro caso o usuário não fornece nenhum argumento e o padrão MINÚSCULO é atendido.

Definição do padrão

```
1 def padronizar_codigos(lista_codigos, padrao='m'):
2     for i, item in enumerate(lista_codigos):
3         item = item.replace(' ', ' ')
4         item = item.strip()
5         if padrao == 'm':
6             item = item.casefold()
7         elif padrao == 'M':
8             item = item.upper()
9         lista_codigos[i] = item
10    return lista_codigos
```

Argumento 'm' omitido

```
12 cod_produtos = ['ABC12', 'abc34', 'AbC37']
13 print(padronizar_codigos(cod_produtos,))
```

```
['abc12', 'abc34', 'abc37']
```

```
12 cod_produtos = ['ABC12', 'abc34', 'AbC37']
13 print(padronizar_codigos(cod_produtos, padrao='M'))
```

```
['ABC12', 'ABC34', 'ABC37']
```

Argumento 'M' fornecido, forçando a não usar o padrão



Módulo 14 – Criando suas funções em Python – Falando um pouco mais dos returns das functions

Como vimos anteriormente o return nos retorna sempre o “resultado” da nossa função.

É possível retornar booleans, integers, floats, variáveis, etc.

No entanto, é muito importante explicar que o return funciona como um FIM de uma função.

Ou seja, se a função chegar até a linha de código RETURN ela irá encerrar a função mesmo que se trate de um FOR como no exemplo ao lado.

Essa função não faz muito sentido de existir, mas podemos ver que apesar de termos definido que nossa função retorna o valor i com um range de 5, apenas o valor 0(PRIMEIRA ITERAÇÃO DO FOR) foi retornado.

Por que? Porque ao chegar ao return, a nossa função chega ao FIM.

```
1 def fazer_for():
2     for i in range(5):
3         return i
4
5 print(fazer_for())
```

Ao chegarmos nessa linha de código, a função será interrompida. Ou seja, só haverá UMA iteração no FOR e não CINCO como definimos no RANGE

0

Retorno esperado: 0,1,2,3,4

Retorno obtido: 0 -> Apenas a primeira iteração do FOR

Módulo 14 – Criando suas funções em Python – Return com mais de uma informação (1/2)

Vamos aprofundar um pouco mais no RETURN.

Nossas funções até agora, em geral, são simples e possuem retornos igualmente simples com apenas um valor por exemplo.

No entanto, é possível que o retorno necessário seja um pouco mais complexo com mais resultados.

Ao lado temos uma função ainda bem simples, mas que nos permite entender o conceito.

Dentro da nossa função **operações_básicas** calculamos resultados como soma, diferença, multiplicação e divisão.

Uma informação importante é que o retorno é uma TUPLA. Isso nos permitirá trabalhar melhor com esses dados.

```
1 def operacoes_basicas(num1,num2):  
2     soma = num1+num2  
3     diferenca = num1-num2  
4     mult = num1*num2  
5     divisao = num1/num2  
6     return (soma, diferenca, mult, divisao)  
7  
8 print(operacoes_basicas(10,2))
```

(12, 8, 20, 5.0)

Ao invés de apenas 1 resultado, temos 4 resultados da nossa função

Retorno de todas as variáveis calculadas dentro da função em uma TUPLA

Módulo 14 – Criando suas funções em Python – Return com mais de uma informação (2/2)

Vamos para um caso um pouco mais complexo.

Nesse caso estamos avaliando quais vendedores bateram a meta.

Podemos perceber que nossa função retorna uma tupla com 2 valores:

- % que bateram a meta (perc_baterammeta);
- Pessoas que bateram a meta (vendedores_acima_media)

Retorno da função será uma TUPLA

% de vendedores que bateram a meta
LISTA de vendedores que bateram a meta

```

1  meta = 10000
2  vendas = {
3      'João':15000,
4      'Julia':27000,
5      'Marcus':9900,
6      'Maria':3750,
7      'Ana':10300,
8      'Alon':7870,
9  }
10
11 def calculo_meta(meta,vendas):
12     bateram_meta = []
13     for vendedor in vendas:
14         if vendas[vendedor]>=meta:
15             bateram_meta.append(vendedor)
16     perc_baterammeta = len(bateram_meta)/len(vendas)
17     return perc_baterammeta,bateram_meta
18
19 p_meta,vendedores_acima_meta = calculo_meta(meta,vendas)
20 print(p_meta)
21 print(vendedores_acima_meta)

```

0.5

['João', 'Julia', 'Ana']

Unpacking a tupla



Módulo 14 – Criando suas funções em Python – Docstrings e annotations (1/2)

Não obrigatoriamente todas as funções criadas serão usadas apenas por você, por isso é importante que elas sejam organizadas e compreensíveis por outras pessoas.

Essa organização passa pelos:

Docstrings : Iniciados por `'''` permitem strings de mais de uma linha. Aqui você pode descrever o que sua função faz, argumentos que ela possui, etc. Você pode criar o seu padrão, mas caso não possua um, temos um exemplo no print ao lado.

Informações podem ser acessadas usando **SHIFT+TAB** quando o cursor estiver na função desejada

```

1 def minha_funcao(arg1,arg2,arg3):
2     ''' O que a minha função faz.
3
4     Meus argumentos:
5     arg1(tipo do meu argumento):Comentário sobre o argumento
6     arg2(tipo do meu argumento):Comentário sobre o argumento
7     arg3(tipo do meu argumento):Comentário sobre o argumento
8
9     return
10    O retorno da minha função
11    '''
12    soma = arg1+arg2+arg3
13    return soma
14
15 minha_funcao()
16

```

Docstring

Signature: `minha_funcao(arg1, arg2, arg3)`
Docstring:
 O que a minha função faz.

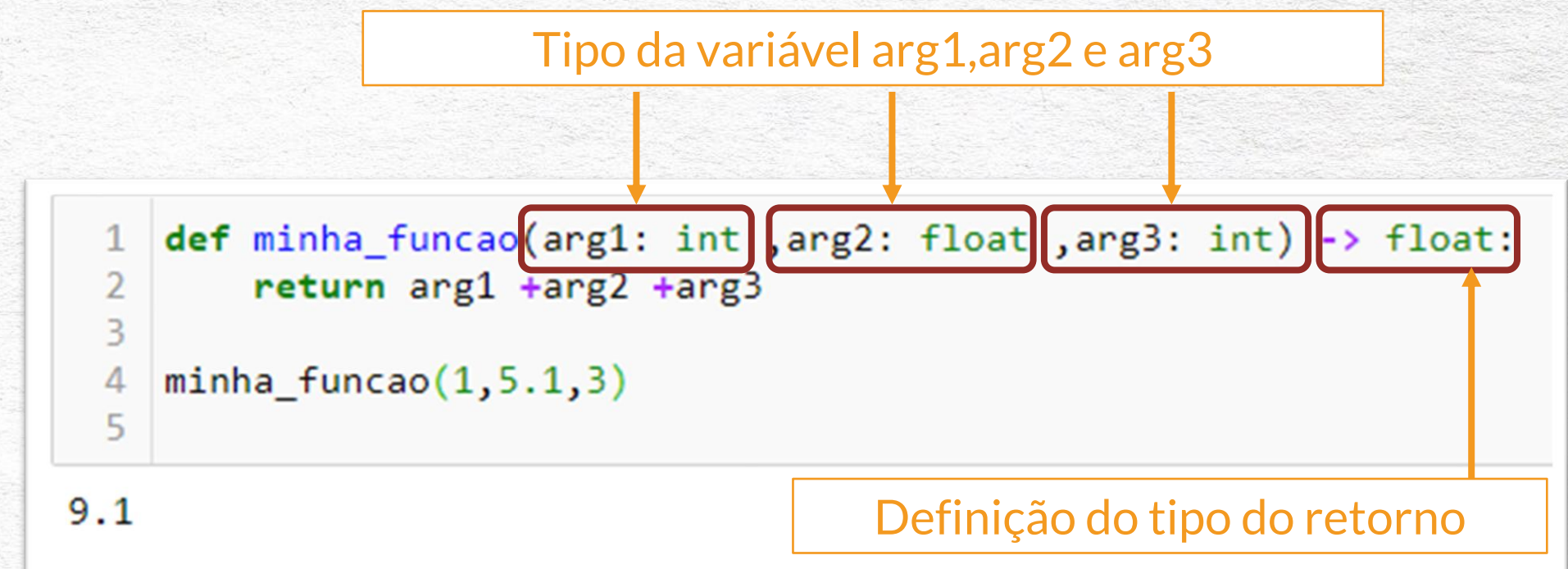
Meus argumentos:
 arg1(tipo do meu argumento):Comentário sobre o argumento
 arg2(tipo do meu argumento):Comentário sobre o argumento
 arg3(tipo do meu argumento):Comentário sobre o argumento

Módulo 14 – Criando suas funções em Python – Docstrings e annotations (2/2)

Não obrigatoriamente todas as funções criadas serão usadas apenas por você, por isso é importante que elas sejam organizadas e compreensíveis por outras pessoas.

Essa organização passa pelos:

Annotations : É uma versão mais simplificada de um docstrings. As informações sobre a função são escritas dentro da própria função ao ser definida.



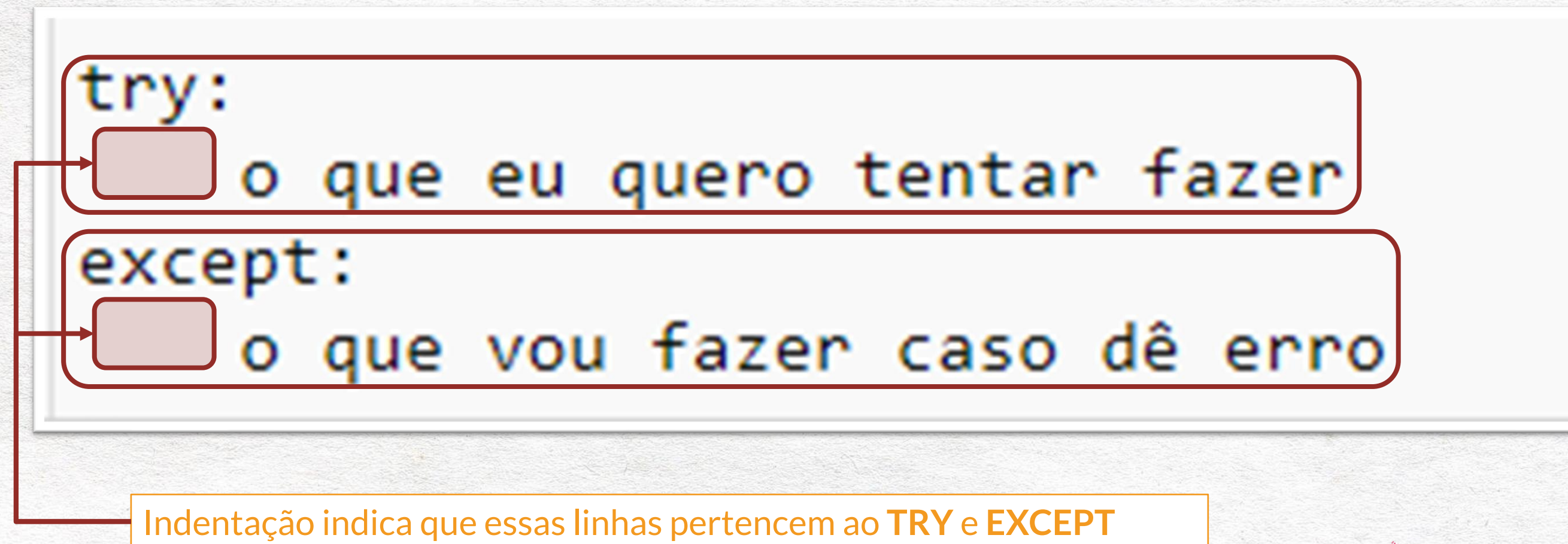
Módulo 14 – Criando suas funções em Python – Exceções e erros em funções (1/4)

Até agora, sempre que nossos códigos acusavam algum erro, significava que seu programa seria encerrado.

No entanto, pode ocorrer uma situação em que o resultado ou o valor inserido pelo usuário está errado, mas você não quer que ele pare nesse momento, e sim que o corrija.

Para esses casos podemos usar o **TRY** e o **EXCEPT**.

SEMPRE que o TRY for utilizado, deverá haver um EXCEPT.



Módulo 14 – Criando suas funções em Python – Exceções e erros em funções (2/4)

Primeiro vamos entender o que são os erros na realidade.

Sabemos que são um pouco assustadores, principalmente no começo. Milhões de linhas de código que não são fáceis de entender.

Vamos pegar um exemplo para analisar e depois vamos voltar para nosso TRY e EXCEPT e como usá-lo.

```
1 print(erro)
```

NameError

Nome do tipo de erro

Traceback (most recent call last)

<ipython-input-5-a751457e33f5> in <module>

----> 1 print(erro)

Indica a linha que o erro foi indicado

NameError: name 'erro' is not defined

Detalhamento do erro

Módulo 14 – Criando suas funções em Python – Exceções e erros em funções (3/4)

Bem, agora que entendemos um pouco mais sobre a estrutura de um erro fornecido pelo Python, vamos usar o Try e Except para criar o nosso próprio erro, que nos permita informar ao usuário.

Vamos criar uma função que indica qual o tipo de e-mail utilizado pelo usuário.

Perceba que no bloco EXCEPT usamos ao invés do return, o **RAISE**. Isso fará com que um erro seja exibido ao usuário.

Veremos isso melhor no próximo slide.

Docstring

```
def descobrir_servidor(email):  
    try:  
        posicao_a = email.index('@')  
    except:  
        raise ValueError('Email digitado não tem @, digite novamente')  
    else:  
        servidor = email[posicao_a:]  
        if 'gmail' in servidor:  
            return 'gmail'  
        elif 'hotmail' in servidor or 'outlook' in servidor or 'live' in servidor:  
            return 'hotmail'  
        elif 'yahoo' in servidor:  
            return 'yahoo'  
        elif 'uol' in servidor or 'bol' in servidor:  
            return 'uol'  
        else:  
            return 'não determinado'
```

email = input('Qual o seu e-mail?')
print(descobrir_servidor(email))

Caso o try seja atendido, entraremos no bloco else

“Chamando” a função descobrir_servidor

Módulo 14 – Criando suas funções em Python – Exceções e erros em funções (4/4)

Bem, agora que entendemos um pouco mais sobre a estrutura de um erro fornecido pelo Python, vamos usar o Try e Except para criar o nosso próprio erro, que nos permita informar ao usuário.

Vamos criar uma função que indica qual o tipo de e-mail utilizado pelo usuário.

Perceba que no bloco EXCEPT usamos ao invés do return, o **RAISE**. Isso fará com que um erro seja exibido ao usuário.

Veremos isso melhor no próximo slide.

```
def descobrir_servidor(email):
    try:
        posicao_a = email.index('@')
    except:
        raise ValueError('Email digitado não tem @, digite novamente')
    else:
```

Qual o seu e-mail?hashtaggmail.com

Usuário digita e-mail sem @

```
ValueError                                Traceback (most recent call last)
<ipython-input-6-e31cf5c841f2> in descobrir_servidor(email)
      2     try:
----> 3         posicao_a = email.index('@')
      4     except:
```

ValueError: substring not found

During handling of the above exception, another exception occurred:

```
ValueError                                Traceback (most recent call last)
<ipython-input-9-882fae42d366> in <module>
      1 email = input('Qual o seu e-mail?')
----> 2 print(descobrir_servidor(email))
```

```
<ipython-input-6-e31cf5c841f2> in descobrir_servidor(email)
      3     posicao_a = email.index('@')
      4     except:
----> 5         raise ValueError('Email digitado não tem @, digite novamente')
      6     else:
      7         servidor = email[posicao_a:]
```

ValueError: Email digitado não tem @, digite novamente

Erro personalizado



Módulo 14 – Criando suas funções em Python – Múltiplos Argumentos para uma Function (1/2)

Voltando para o nosso exemplo de cálculo de média das notas. No caso anterior criamos 2 argumentos **nota1** e **nota2**, existe uma outra forma que podemos fazer essa declaração de variáveis.

Você verá em alguns lugares durante pesquisas o termo ***args**.

Usando o ***** antes da **variável** indicamos que não estamos tratando de um argumento único e sim um argumento que poderá receber um número ilimitado de valores.

Podemos ver no exemplo que ao usarmos ***notas** permitimos não só ter apenas 2 notas mas 3, 4, 5 ou 10...

Ao invés de limitar o número de notas, criamos um argumento de tamanho variável

```

1 def calcular_media(*notas):
2     soma=0
3     for nota in notas:
4         soma += nota
5     return soma/(len(notas))
6
7 print(calcular_media(10,0,5,6,7,8,9,1,0,4,8,6,4,8,9,4,2,2,2,3,4,5,8,9))
8 print(calcular_media(10,0))
9
5.166666666666667
5.0

```

Apenas 2 argumentos

Diversos argumentos



Módulo 14 – Criando suas funções em Python – Múltiplos Argumentos para uma Function (2/2)

Como vimos, temos tanto argumentos de posição como argumentos **Keywords** (palavras-chave).

O conceito é exatamente o mesmo usado no exemplo anterior. Ao usarmos o ****** antes da **palavra-chave**, temos uma função que aceita receber diversas palavras chaves na função.

Esse conceito é comumente visto com ****kwargs**.

Ao lado temos um exemplo deste caso. Aqui o usuário poderá (ou não) passar informações para que influenciem no preço final do produto.

No caso dos ****kwargs**, é importante entender o que a função faz. Olhando dentro da nossa função `preco_final`, vemos que existem três variáveis que influenciam o preço:

- desconto;
- garantia_extra;
- Imposto.

****** indicando que temos Keywords variáveis

```
1 def preco_final(preco, **adicionais):
2     print(adicionais)
3     if 'desconto' in adicionais:
4         preco *= (1 - adicionais['desconto'])
5     if 'garantia_extra' in adicionais:
6         preco += adicionais['garantia_extra']
7     if 'imposto' in adicionais:
8         preco *= (1 + adicionais['imposto'])
9     return preco
```

```
1 print(preco_final(1000, desconto=0.1, imposto=0.3))
{'desconto': 0.1, 'imposto': 0.3}
1170.0
```

Perceba que mesmo sem usar todos os argumentos nosso valor final foi calculado.



Módulo 14 – Criando suas funções em Python – Ordem dos Tipos de Argumento

Ok. Entendi que tenho liberdade de ter `*args` ou `**kwargs`, mas por enquanto só vimos isso separadamente.

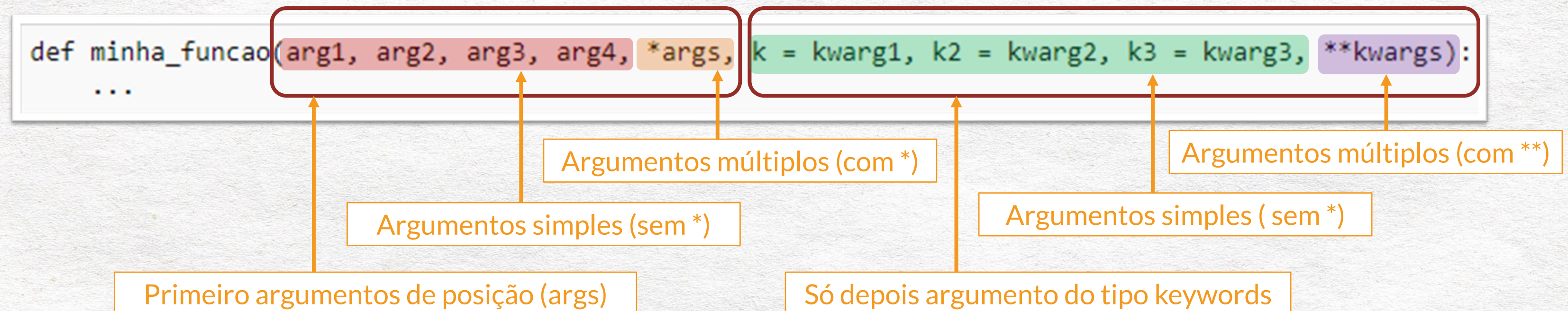
E se eu tiver um caso mais confuso com 3 kwargs, 2 args?

Tem um jeito certo de fazer? Ou é só sair colocando?

Temos 2 regras para seguir:

- 1) Primeiro colocamos os argumentos de **posição** (`args`), depois os de **palavra chave** (`kwargs`)
- 2) Seguindo a ordem anterior, primeiro fazemos os argumentos **simples** (sem `*` ou `**`) e depois os **mútiplos** (com `*` ou `**`)

Vamos dar uma olhada na estrutura abaixo para entendermos melhor.





PYTHON

Módulo 15

LIST COMPREHENSION



Módulo 15 – List Comprehension – O que é List Comprehension (1/3)

O que você aprenderá nesse módulo não é algo OBRIGATÓRIO. Esse módulo tem por objetivo apresentar práticas comuns na construção de códigos em Python. No início, mais importante que escrever seus códigos usando o List Comprehension é saber entender um código que esteja escrito dessa forma.

Por que?

Boa parte do desafio em se programar, seja em Python ou qualquer outra linguagem é saber:

- O que pesquisar;
- Como pesquisar;
- Entender o que se achou;
- Adequar o que já existe para seu projeto.

É muito comum que programadores mais antigos se usem desses artifícios para que seu código fique mais elegante. Por isso é muito importante entender o que são e quais são os “pulos” de código que são usados no mercado.

Módulo 15 – List Comprehension – O que é List Comprehension (2/3)

Vamos para um caso que queremos calcular os impostos de produtos distintos.

O imposto é fixado em 30% do valor do produto.

O que faremos aqui é apresentar 2 soluções distintas que estão CORRETAS, mas escritas de forma distintas.

A primeira utiliza a lógica que temos usado até aqui (TRADICIONAL), já a segunda, usa o conceito de list comprehension.

Abaixo temos as duas formas de se resolver esse problema.

```
preco_produtos = [100, 150, 300, 5500]
produtos = ['vinho', 'cafeiteira', 'microondas', 'iphone']
```

“Tradicional”

```
1 impostos = []
2 for item in preco_produtos:
3     impostos.append(item * 0.3)
4 print(impostos)
```

```
[30.0, 45.0, 90.0, 1650.0]
```

List Comprehension

```
1 impostos = [preco * 0.3 for preco in preco_produtos]
2 print(impostos)
```

```
[30.0, 45.0, 90.0, 1650.0]
```

Módulo 15 – List Comprehension – O que é List Comprehension (3/3)

Você é capaz de entender o que está sendo feito no list comprehension comparando os dois códigos? Achou estranho? Fique tranquilo, não é trivial no início. Vamos entender melhor.

```
preco_produtos = [100, 150, 300, 5500]  
produtos = ['vinho', 'cafeiteira', 'microondas', 'iphone']
```

```
1 impostos = []  
2 for item in preco_produtos:  
3     impostos.append(item * 0.3)  
4 print(impostos)
```

```
[30.0, 45.0, 90.0, 1650.0]
```

```
1 impostos = [preco * 0.3 for preco in preco_produtos]  
2 print(impostos)
```

```
[30.0, 45.0, 90.0, 1650.0]
```

Essencialmente o que ocorre aqui é:

- Na primeira solução (“Tradicional”):
Criamos uma variável item que percorre a lista preço_produtos, que explicitamente adiciona(.append) o valor calculado na lista impostos.
- Na segunda solução :
O método .append não é explícito. A leitura da linha de código seria: “para cada preço da lista preco_produtos será calculado um item da lista imposto. Esse valor será calculado por preço * 0.3”



Módulo 15 – List Comprehension – Exemplo de List Comprehension

Vamos para outro exemplo.

Aqui, temos 2 listas separadas mas que se relacionam entre si.

Sabemos que se por exemplo ordenarmos a lista de vendas_produtos do maior para o menor nada acontecerá na nossa lista produtos.

Ou seja, as posições que antes estavam “casadas” agora estão distintas.

Antes de qualquer tratamento, precisamos UNIR essas duas listas. Uma boa forma é transformar essas duas listas em uma única lista de tuplas.

Para isso, usaremos a função ZIP() que já vimos anteriormente no módulo de dicionários.

Possuindo a lista de tuplas é só aplicarmos o unpacking usando o List Comprehension como apresentado na figura ao lado.

```
1 vendas_produtos = [1500, 150, 2100, 1950]
2 produtos = ['vinho', 'cafeiteira', 'microondas', 'iphone']
3
4 lista_aux = list(zip(vendas_produtos, produtos))
5 lista_aux.sort(reverse=True)
6 produtos = [produto for vendas, produto in lista_aux]
7 print(produtos)
```

['microondas', 'iphone', 'vinho', 'cafeiteira']

List comprehension que pode ser lida como:
“para cada item venda e produto da lista_aux, será criado um item produto na lista produtos”

Módulo 15 – List Comprehension – List Comprehension filtrando com if

Mais um caso onde o list comprehension nos auxilia a reduzir as linhas de código. Este caso é ainda menos intuitivo, principalmente no início da jornada na programação.

LEMBRANDO, nada disso é obrigatório, são formas distintas de se escrever onde o resultado final é o mesmo.

```
1 meta = 1000
2 vendas_produtos = [1500, 150, 2100, 1950]
3 produtos = ['vinho', 'cafeiteira', 'microondas', 'iphone']
```

```
1 produtos_acima_meta = [produto for i, produto in enumerate(produtos) if vendas_produtos[i] > meta]
2 print(produtos_acima_meta)

['vinho', 'microondas', 'iphone']
```

Nova lista que receberá os produtos que estão acima da meta

Valor que será incluído na lista "produtos_acima_meta"

For que permite percorrer toda a lista produtos

Condição a ser atendida.

Módulo 15 – List Comprehension – Tratando casos em List Comprehension

Mais um caso onde o list comprehension nos auxilia a reduzir as linhas de código.
Nesse caso, vamos usar as estruturas de IF/ELSE para criar uma lista de vendedores que bateram uma meta pré-definida.

LEMBRANDO, nada disso é obrigatório, são formas distintas de se escrever onde o resultado final é o mesmo.

```
1 vendedores_dic = {'Maria': 1200, 'José': 300, 'Antônio': 800, 'João': 1500, 'Francisco': 1900, 'Ana': 2750, 'Luiz': 400, 'Pa
2 meta = 1000
```

```
1 bonus = [vendedores_dic[item] * 0.1 if vendedores_dic[item] > meta else 0 for item in vendedores_dic]
2 print(bonus)

[120.0, 0, 0, 150.0, 190.0, 275.0, 0, 0, 0, 0, 0, 0, 110.0, 0, 0, 0, 0, 0, 111.10000000000001, 0, 0, 0, 0]
```

Nova lista que
receberá os
produtos que estão
acima da meta

Valor que será incluído na
lista "bonus"

Condição a ser atendida

Caso a condição não
seja atendida

For que percorre os
itens do dicionário
"vendedores_dic"

Módulo 15 – List Comprehension – List Comprehension não serve apenas para criar listas

Mais um caso onde o list comprehension nos auxilia a reduzir as linhas de código.

Nesse caso não vamos criar uma lista, mas sim retornar um valor da soma das vendas dos produtos listados como “TOP5”.

LEMBRANDO, nada disso é obrigatório, são formas distintas de se escrever onde o resultado final é o mesmo.

```
1 produtos = ['coca', 'pepsi', 'guarana', 'skol', 'brahma', 'agua', 'del valle', 'dolly', 'red bull', 'cachaça', 'vinho tinto']
2 vendas = [1200, 300, 800, 1500, 1900, 2750, 400, 20, 23, 70, 90, 80, 1100, 999, 900, 880, 870, 50, 1111, 120, 300, 450, 800]
3 top5 = ['agua', 'brahma', 'skol', 'coca', 'leite de castanha']
```

```
1 total_top5 = sum(vendas[i] for i, produto in enumerate(produtos) if produto in top5)
2 print(total_top5)
3 print('Top 5 representou {:.1%} das vendas'.format(total_top5/sum(vendas)))
```

8461

Top 5 representou 50.6% das vendas

Variável que
receberá a SOMA
das vendas dos top5
produtos

Função
SOMA.

Valores que
serão somados
CASO a condição
seja atendida

Percorre a lista
produtos testando a
condição definida
na parte final da
linha

Testa a condição se o produto
da lista produtos da vez está na
lista top5.





PYTHON

MÓDULO 17

MÓDULOS, BIBLIOTECAS E INTRODUÇÃO À ORIENTAÇÃO A OBJETO



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – O que é Orientação a Objeto e Por que isso importa

É bem provável que você já tenha escutado a expressão “orientado a objetos”. Assim como outras linguagens de programação, o Python também é orientado a objetos.

E o que isso significa?

Vamos primeiro usar um exemplo da vida real

O que pode ser feito com um objeto como o Celular?
Ligar, mandar mensagens, ver vídeos, etc

Com uma bicicleta?
Andar, empurrar, etc

Na programação, essas ações que podem ser executadas com os objetos são chamados de **Métodos**. Então para cada um dos objetos que temos é possível fazer algumas ações.

A gente vai ver isso melhor mais para frente. Fica tranquilo!



Ligar
Mandar Mensagens
Ver vídeos



Andar
Empurrar
Frear

Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – O que são módulos e qual a importância

Até agora só vimos os objetos e métodos que existem dentro do próprio Python, mas uma das principais “armas” dessa linguagem são as chamadas bibliotecas.

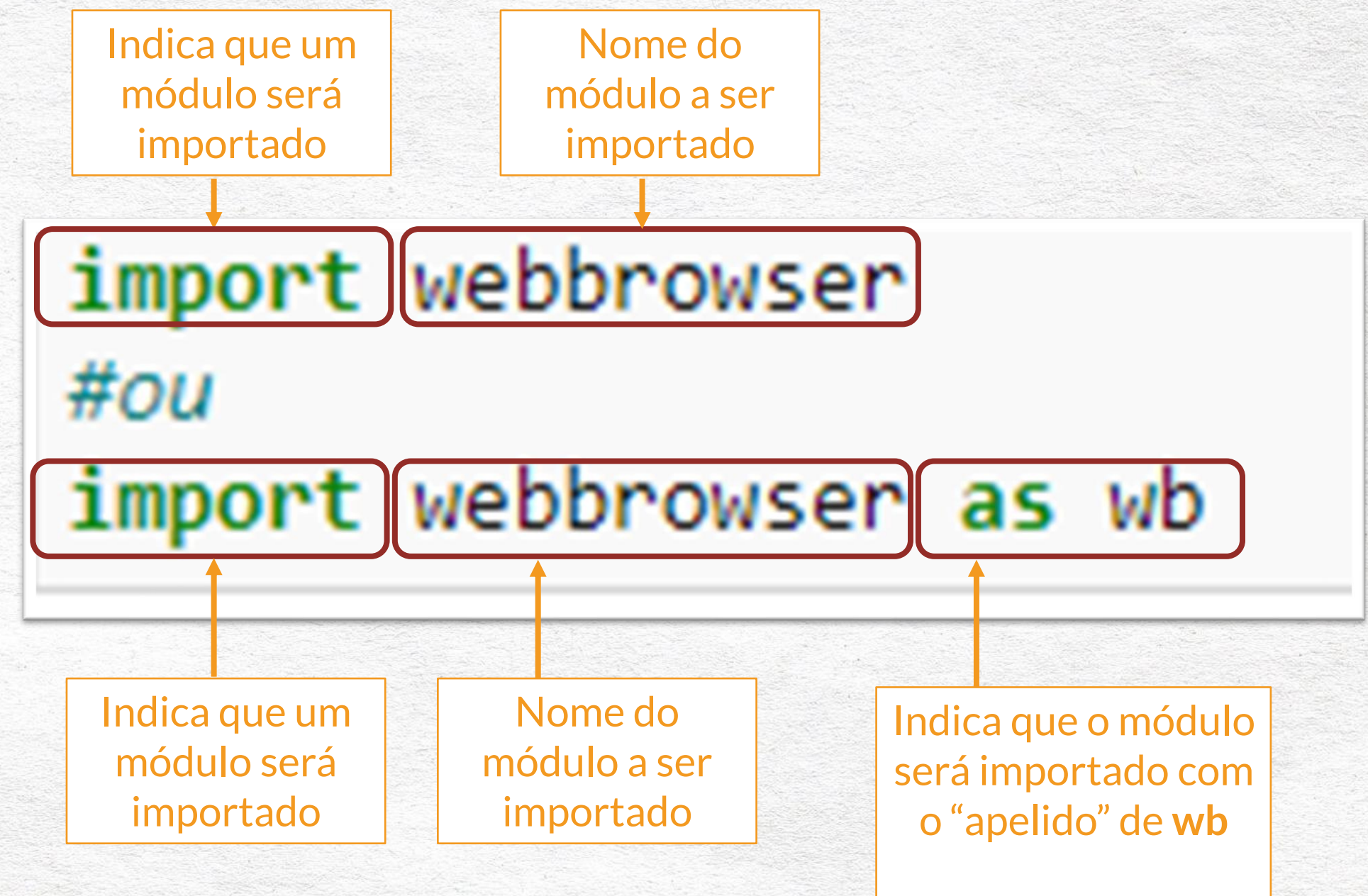
O que são bibliotecas? Elas são módulos prontos que podem ser importados para nossos projetos. Elas nos permitem executar códigos que nos demandariam muito tempo de desenvolvimento para executá-los.

Existem milhares de módulos disponíveis, ter todos eles integrados no Python nos demandaria muita memória.

Por isso, vamos precisar, sempre que necessário, importar esses módulos dentro de nossos projetos.

Para essa importação usaremos o comando:

import



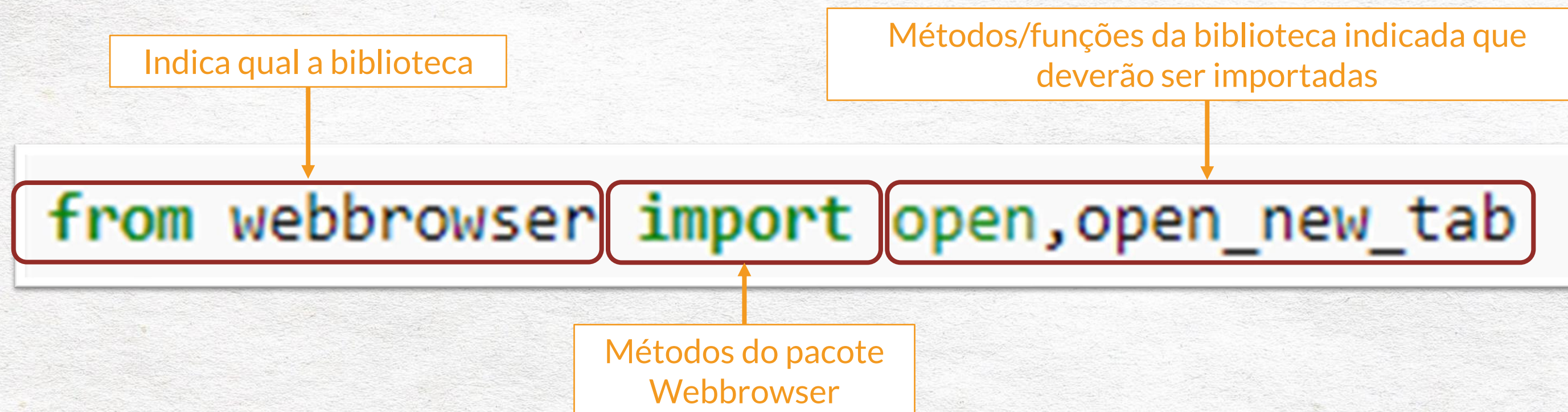
Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – O que são módulos e qual a importância

Em alguns casos não é interessante importar toda a biblioteca mas apenas uma parte dela.

Para isso vamos usar a estrutura abaixo:

From biblioteca import módulo

Vamos ver um exemplo ainda usando o webbrowser.



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Como ler um Módulo

Muito provavelmente o que você está se perguntando é:

- 1) Mas o que faz o webbrowser?
- 2) Como saber que o módulo que preciso é o webbrowser?

É impossível apresentarmos no curso todas as bibliotecas existentes.

O webbrowser é uma das milhares bibliotecas existentes. Essa, especificamente nos ajuda a interagir com a internet (abrir uma página Web por exemplo).

No curso vamos apresentar as principais/mais famosas, mas uma ótima forma de descobrir bibliotecas é usando o nosso amigo GOOGLE.

A comunidade do Python é global e conseguimos achar resposta para qualquer dúvida que a gente venha a ter. Além disso, estamos aqui também para te ajudar OBVIAMENTE !😊

Browser Controller Objects

Browser controllers provide these methods which parallel three of the module-level convenience functions:

`controller.open(url, new=0, autoraise=True)`

Display *url* using the browser handled by this controller. If *new* is 1, a new browser window is opened if possible. If *new* is 2, a new browser page ("tab") is opened if possible.

`controller.open_new(url)`

Open *url* in a new window of the browser handled by this controller, if possible, otherwise, open *url* in the only browser window. Alias `open_new()`.

`controller.open_new_tab(url)`

Open *url* in a new page ("tab") of the browser handled by this controller, if possible, otherwise equivalent to `open_new()`.

Métodos do pacote
Webbrowser



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Módulo time

Vamos aprofundar em um módulo bem recorrente em projetos de Python.

O módulo **time** é um dos vários módulos que nos ajudam a trabalhar com tempo e datas, mas é uma das mais famosas.

Essa função, possui diversos métodos. Aqui, vamos ver algumas delas mas caso você tenha interesse em conhecer mais sobre ela você pode acessar a documentação no link abaixo:

<https://docs.python.org/3/library/time.html>

Vamos primeiro, importar nossa biblioteca

Usando o **import time**

```
1 import time
```



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Módulo time

Vamos ver os principais métodos da biblioteca **time**:

- **EPOCH** – É o marco zero do Python. 01/01/1970;
- **time ()** – Nos retorna a diferença entre o tempo de hoje até a EPOCH em segundos
- **ctime()** – retorna uma **string** com a data no modelo UTC. UTC é um formato padrão utilizado pelo Python;
- **sleep()** – faz com que o python aguarde um tempo dado (em segundos) para executar a próxima linha de código;
- **gmtime()** – retorna as informações de data de forma detalhada;

Perceba que para usarmos os métodos sempre chamamos primeiramente a biblioteca.

Ou seja **time.método**

```
1 segundos_hoje = time.time()
2 print(segundos_hoje)
```

1602013004.6264832

```
1 #para esperar 5 segundos fazemos:
2 print('Começando')
3 time.sleep(5)
4 print('Rodou 5 segundos após')
```

Começando
Rodou 5 segundos após

```
1 data_hoje = time.ctime()
2 #ou entao data_hoje = time.ctime(time())
3 print(data_hoje)
```

Tue Oct 6 16:37:36 2020

Métodos

```
1 data_atual = time.gmtime()
2 print(data_atual)
```

time.struct_time(tm_year=2020, tm_mon=10, tm_mday=6, tm_hour=19, tm_min=42, tm_sec=38, tm_wday=1, tm_yday=280, tm_isdst=0)



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Módulo de Gráficos – Exibindo Gráficos no Python

Vamos para outra biblioteca, dessa vez para criação de gráficos. Uma das bibliotecas mais famosas é o **matplotlib**. Caso tenha mais interesse sobre a biblioteca, aqui está o link da documentação da biblioteca:

<https://matplotlib.org/stable/contents.html#>

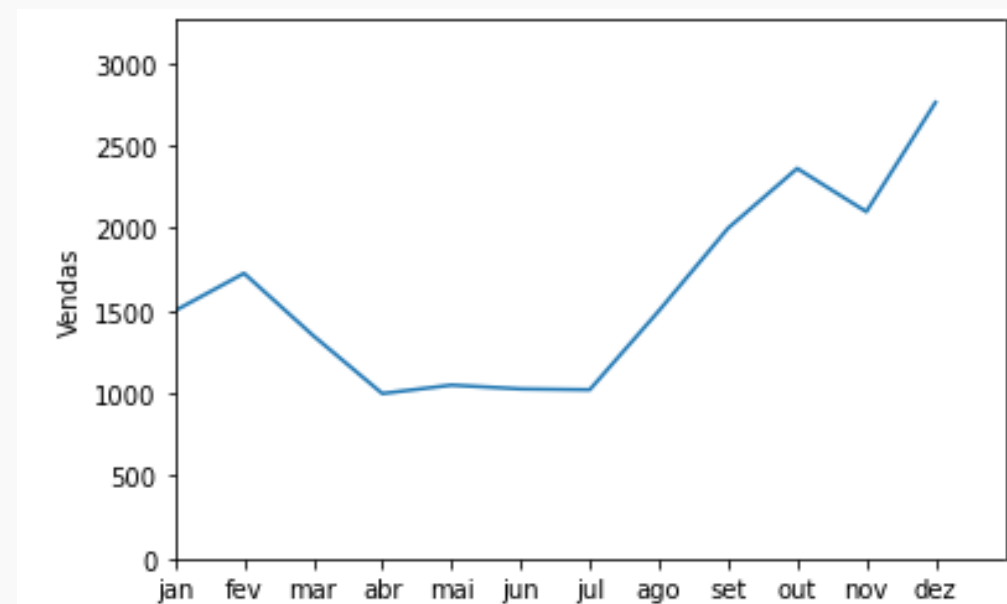
Vamos importar o **matplotlib** como **plt**. Para criarmos um gráfico basta usarmos os métodos abaixo:

- **.plot(eixox,eixoy)** – indica quais são os dados que formarão os gráfico;
- **.ylabel()** – Define o rótulo de dados do eixo Y;
- **.xlabel()** – Define o rótulo de dados do eixo X;
- **.axis()** – Define mínimo e máximo dos eixos X e Y. Nessa ordem.
- **.show()** – Plota o gráfico

```

1 vendas_meses = [1500, 1727, 1350, 999, 1050, 1027, 1022, 1500, 2000, 2362, 2100, 2762]
2 meses = ['jan', 'fev', 'mar', 'abr', 'mai', 'jun', 'jul', 'ago', 'set', 'out', 'nov', 'dez']
3
4 #plotar o gráfico da forma mais simples
5 import matplotlib.pyplot as plt
6
7 plt.plot(meses, vendas_meses)
8 plt.ylabel('Vendas')
9 plt.xlabel('Meses')
10 plt.axis([0, 12, 0, max(vendas_meses)+500])
11 plt.show()
```

Métodos



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Módulo de Gráficos – Exibindo Gráficos no Python

Vamos entender um pouco melhor como são usados este métodos.

O método `plot`. Ao usarmos a ordem (`meses`, `vendas_meses`), definimos que:

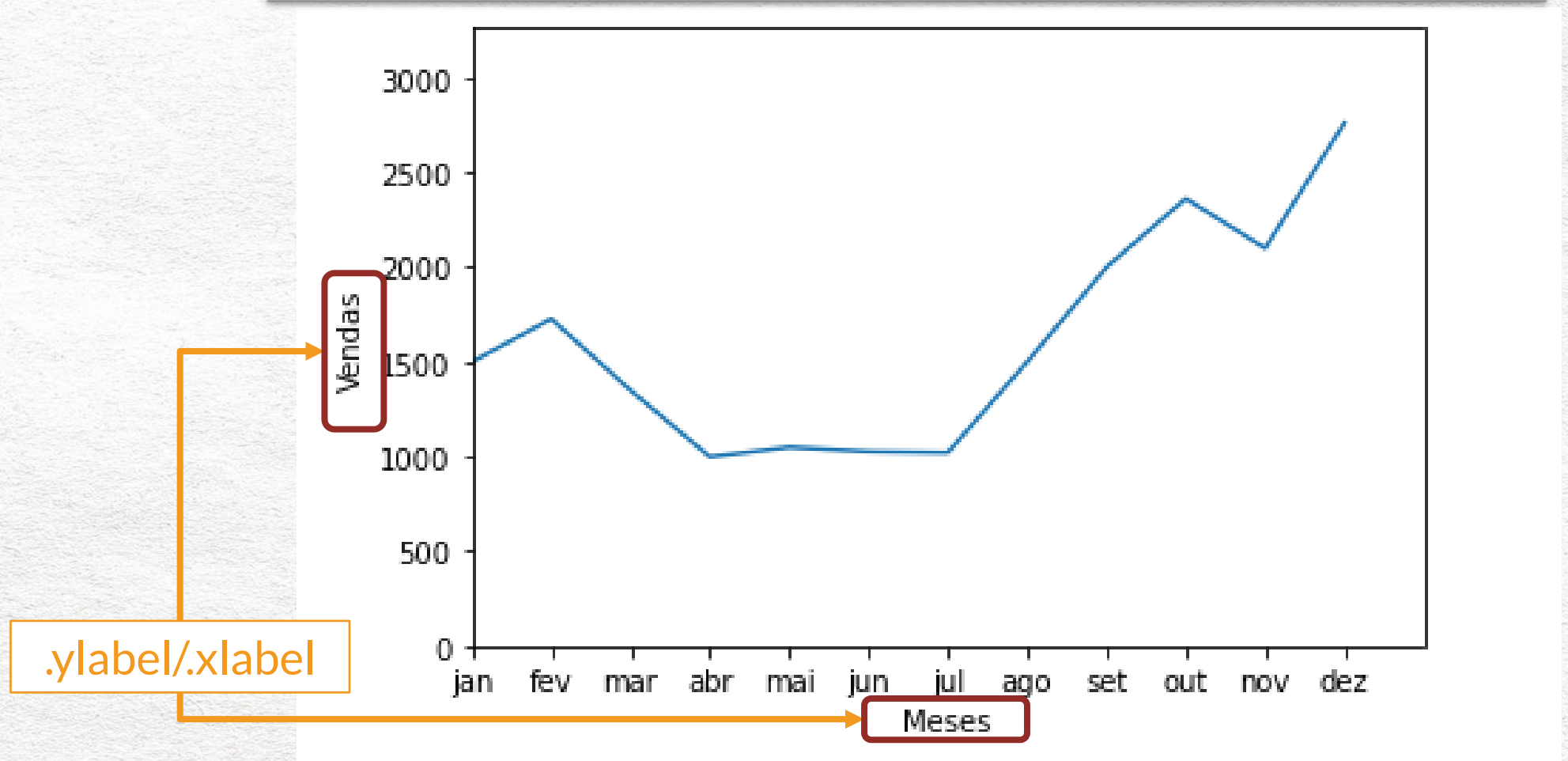
- `meses` estará no eixo X;
- `venda_meses` no eixo y;

Isso ocorre pois os argumentos deste método estabelece essa ordem.

Assim como vimos no módulo de funções, precisamos sempre lembrar que a ordem é importante.

Já os métodos `.ylabel` e `.xlabel` funcionam da mesma forma, só alterando o eixo a que se referem. Respectivamente eixo y e eixo x. Perceba que '`Vendas`' e '`Meses`', são **STRINGS** e não se referem as variáveis `meses` e `vendas_meses`. São apenas rótulos do gráfico.

```
6  
7 plt.plot(meses, vendas_meses)  
8 plt.ylabel('Vendas')  
9 plt.xlabel('Meses')  
10 plt.axis([0, 12, 0, max(vendas_meses)+500])  
11 plt.show()
```



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Módulo de Gráficos – Exibindo Gráficos no Python

O próximo método utilizado é o `.axis()`. Este método nos permite definir os valores mínimos e máximo dos eixos x e y.

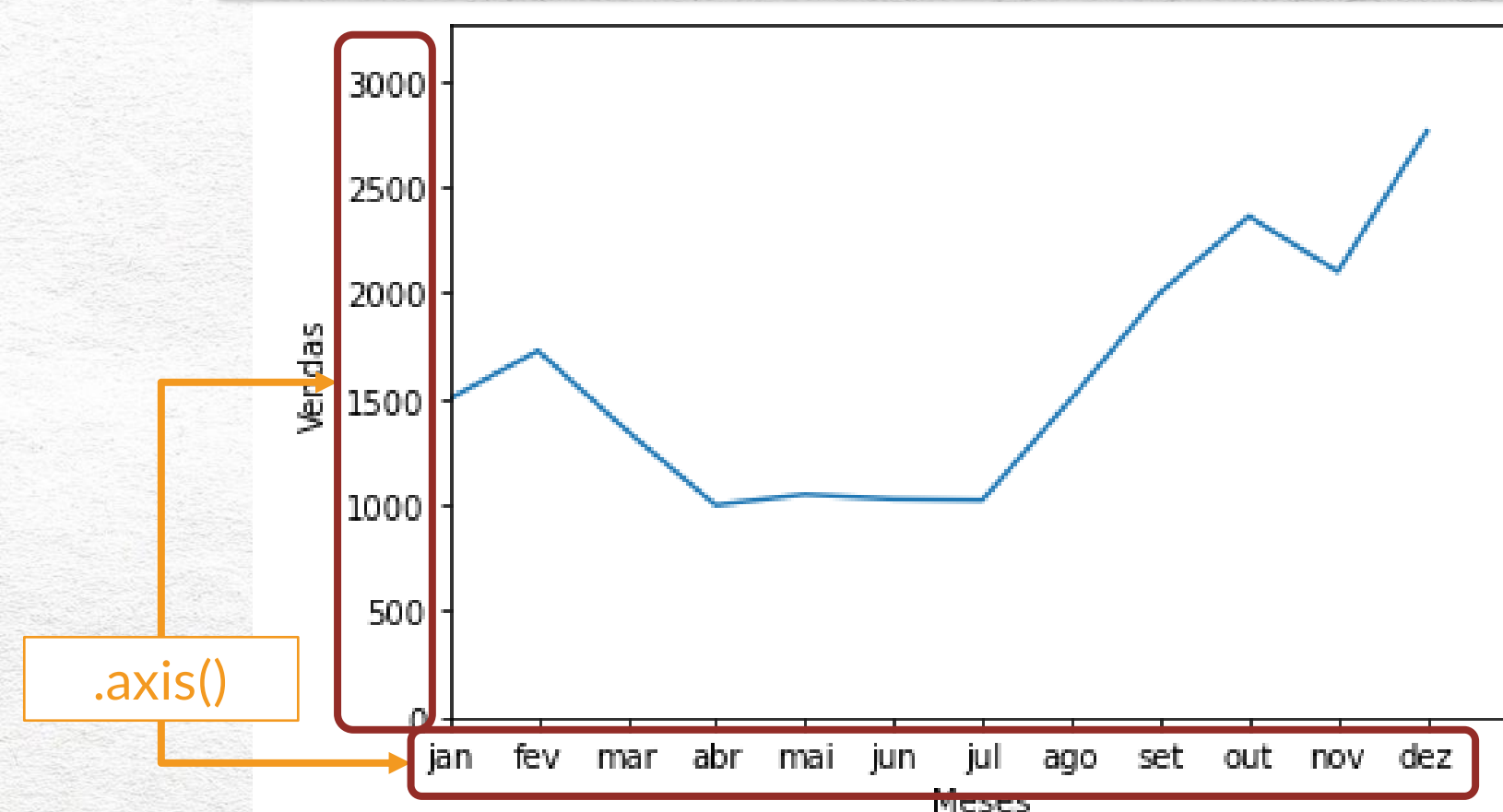
IMPORTANTE! Assim, como vimos no módulo de definição de funções, este método segue uma ordem específica.

- 1) Min eixo x;
- 2) Max eixo x;
- 3) Min eixo y;
- 4) Max eixo y.

Perceba que usamos a própria variável `vendas_meses` como parâmetro máximo do nosso eixo y ao fazermos `max(vendas_meses)+500`.

Por fim, usamos o método `show()` que nos fornece o gráfico definido por nós.

```
6  
7 plt.plot(meses, vendas_meses)  
8 plt.ylabel('Vendas')  
9 plt.xlabel('Meses')  
10 plt.axis([0, 12, 0, max(vendas_meses)+500])  
11 plt.show()
```



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Mais edições de Gráficos e Módulo Numpy

Outro método muito comum, principalmente quando estamos tratando dados estatisticamente, é o numpy. Segue abaixo o link da documentação: https://numpy.org/doc/stable/user/tutorials_index.html

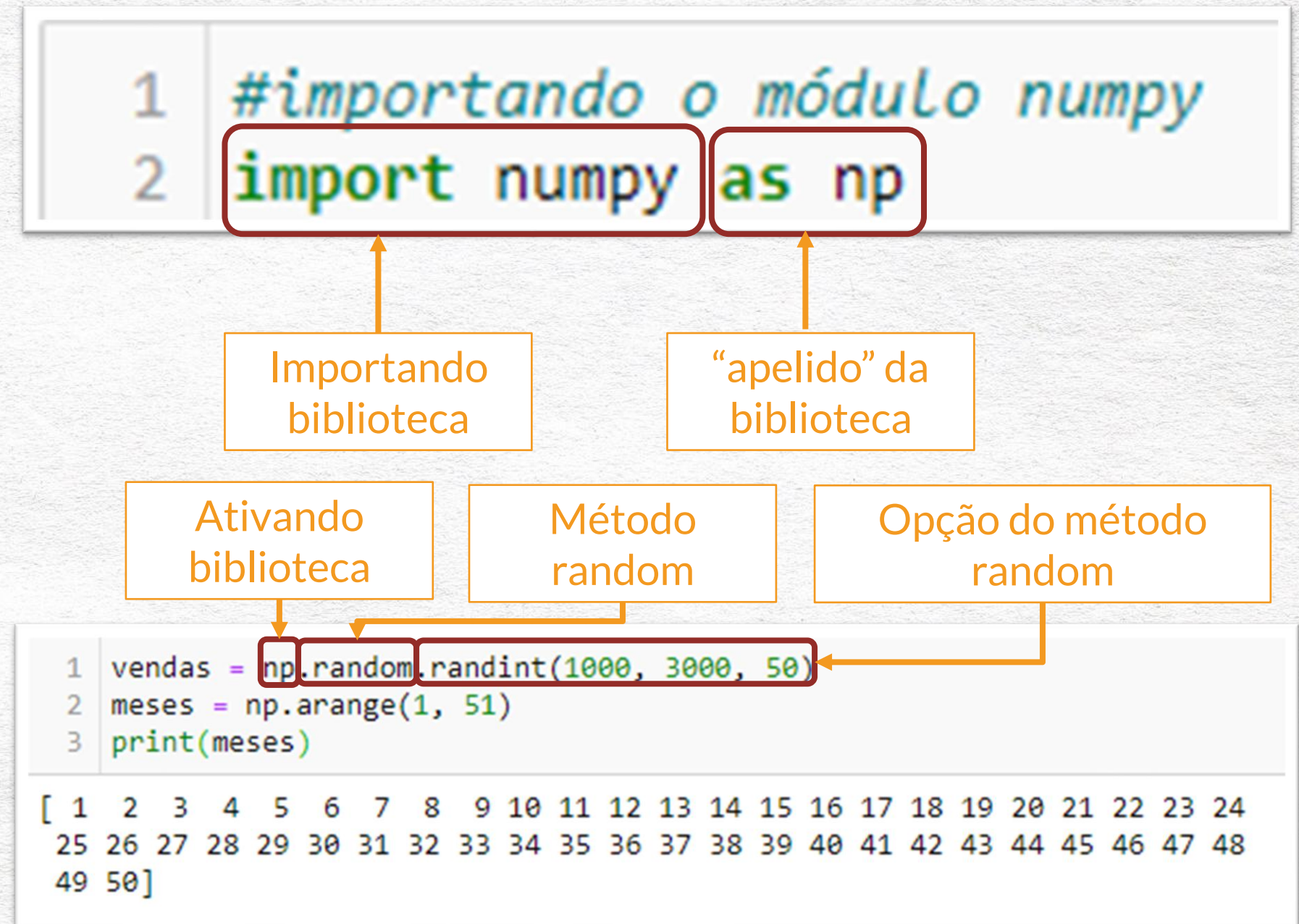
Vamos usar o Numpy juntamente do Matplotlib que vimos anteriormente.

Para importar o Numpy vamos usar o comando abaixo:

Import numpy as np

Aqui vamos replicar o exemplo anterior, mas utilizando o numpy para gerar números aleatórios de vendas (linha 1).

Já na linha 2 vamos usar o método `.arange()` para criação de 50 meses que coreesponderão as vendas geradas aleatoriamente.



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Mais edições de Gráficos e Módulo Numpy

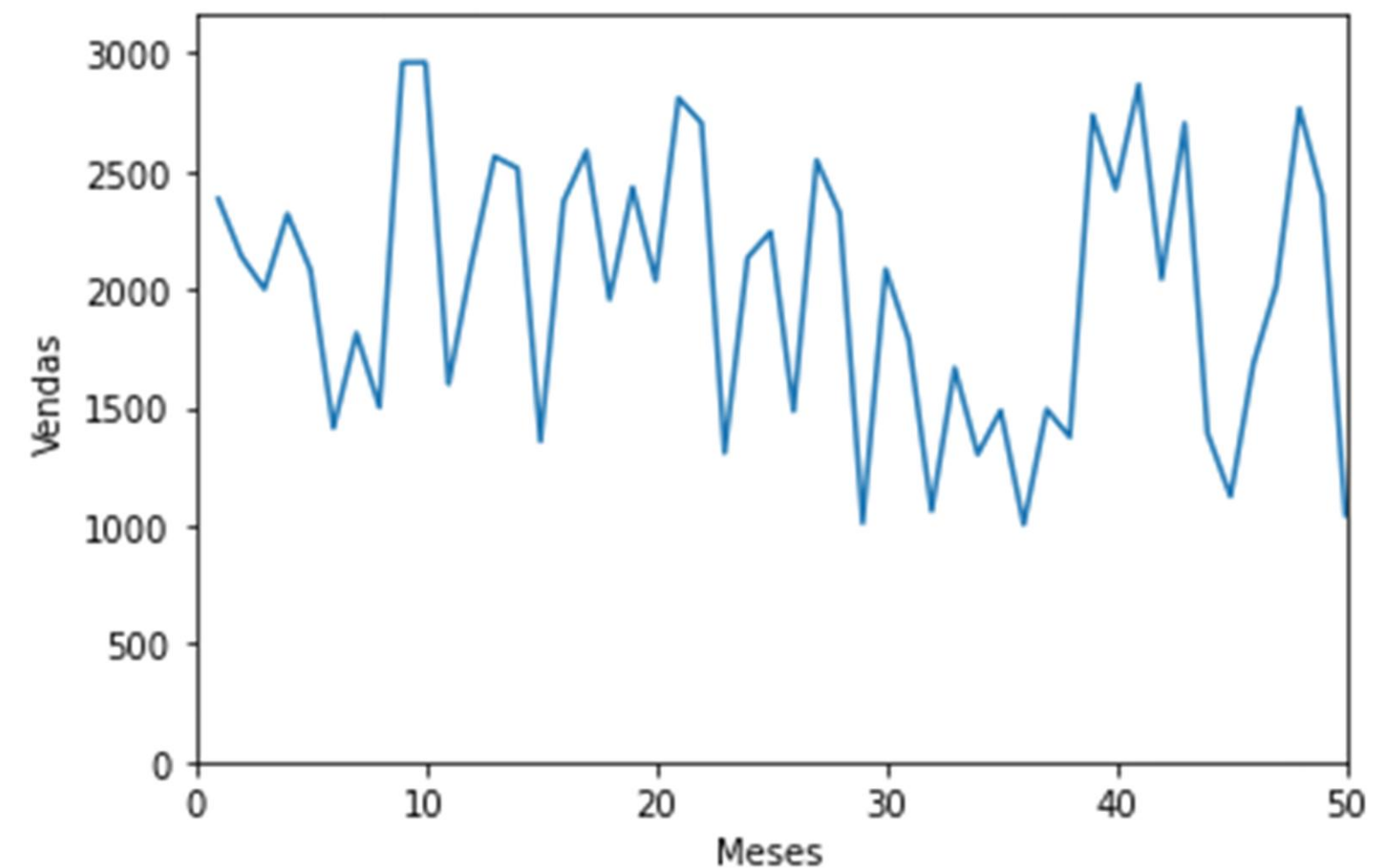
Vamos agora usar outras funcionalidades da nossa biblioteca matplotlib.

Usando a mesma linha de código do gráfico anterior temos este gráfico apresentado ao lado.

No entanto, conforme falamos, nossa biblioteca tem dezenas, centenas de funcionalidades que podemos usar para melhor a apresentação do gráfico.

É isso que vamos fazer nas próximas páginas.

```
1 plt.plot(meses, vendas)
2 plt.axis([0, 50, 0, max(vendas)+200])
3 plt.xlabel('Meses')
4 plt.ylabel('Vendas')
5 plt.show()
```



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Mais edições de Gráficos e Módulo Numpy

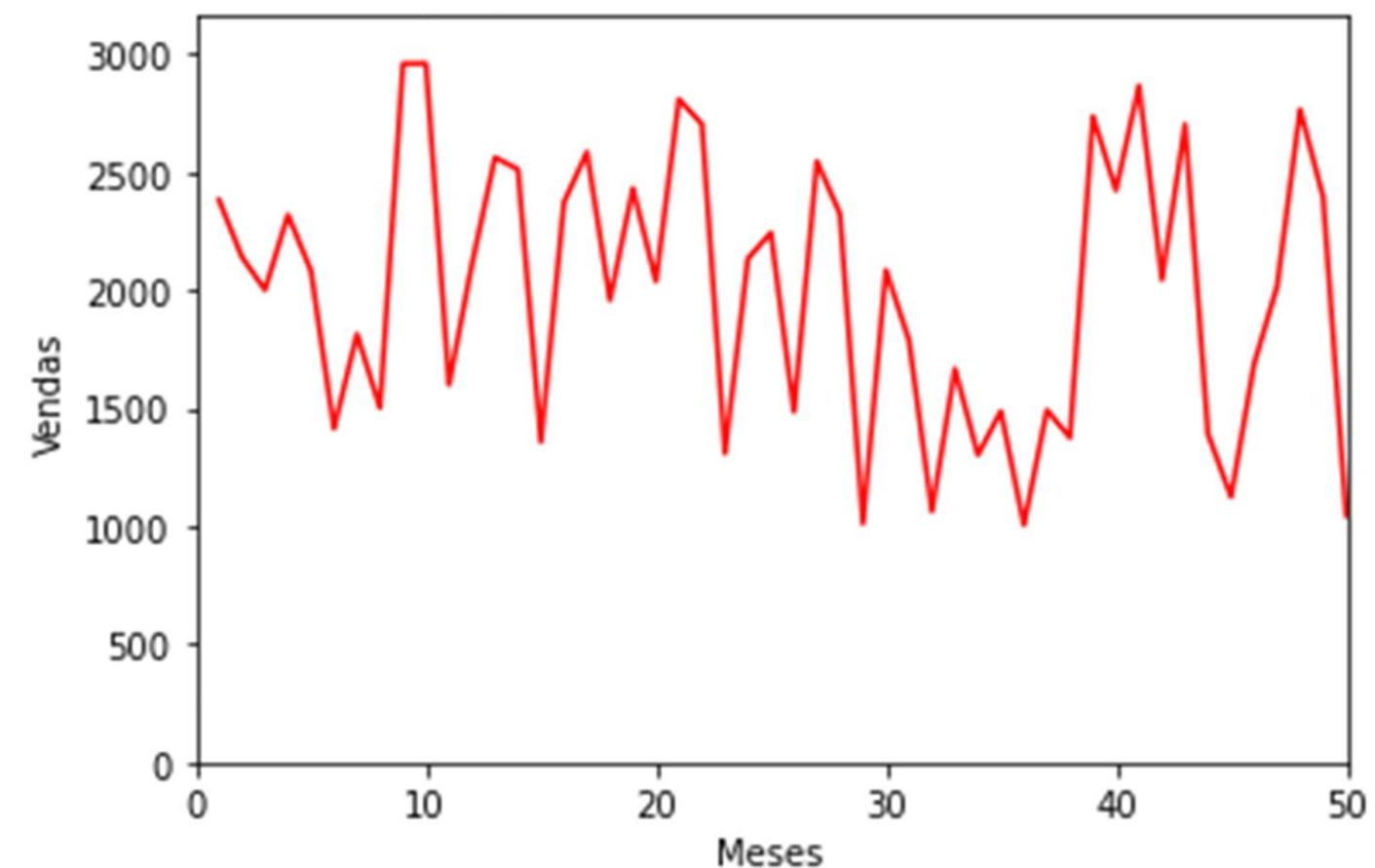
Começando pelo método `.plot()`, vamos alterar a cor da linha do gráfico usando a **keyword** `color =`.

Perceba que apenas acrescentando uma keyword nos argumentos podemos alterar a cor.

Lembre-se!! Ao fazermos isso, estamos alterando a cor padrão (azul). Muito importante também que só foi possível usar a palavra chave **RESPEITANDO** que este argumento só foi utilizado **APÓS** a definição dos eixos X e Y.

```
1 #mudando a linha para apenas os marcadores
2 plt.plot(meses, vendas, color='red')
3 plt.axis([0, 50, 0, max(vendas)+200])
4 plt.xlabel('Meses')
5 plt.ylabel('Vendas')
6 plt.show()
```

Argumento
keyword
(kwarg)



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Mais edições de Gráficos e Módulo Numpy

Aqui são inúmeras as possibilidades de alteração.

Talvez você se pergunte:

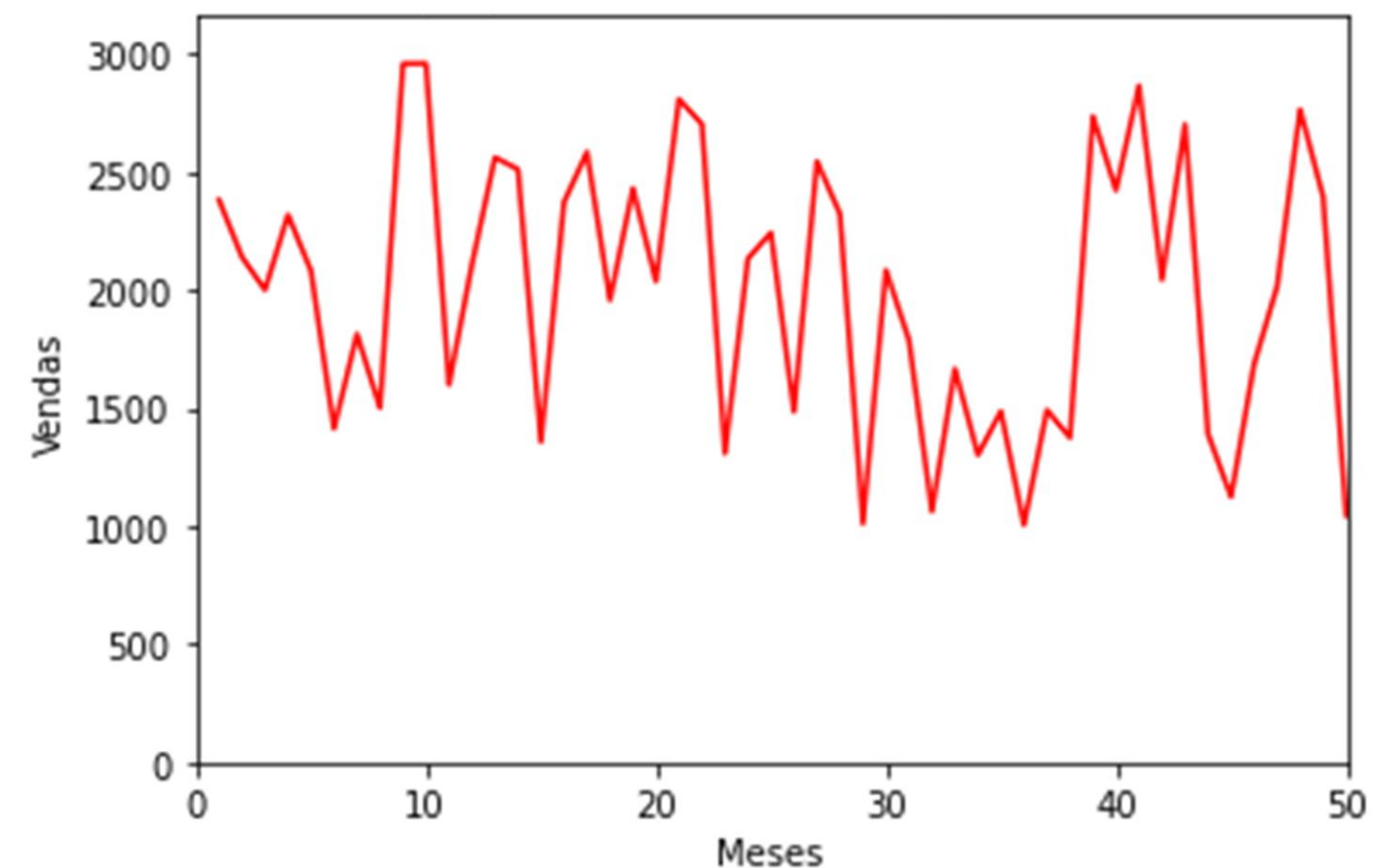
Preciso decorar isso tudo??

A resposta é NÃO! As documentações existem para serem consultadas. Obviamente quanto mais você exercitar, mais você saberá executar sem pesquisar mas **FAZ PARTE** do processo usar as documentações durante a programação.

Seja ela em Python ou qualquer linguagem.

```
1 #mudando a linha para apenas os marcadores
2 plt.plot(meses, vendas, color='red')
3 plt.axis([0, 50, 0, max(vendas)+200])
4 plt.xlabel('Meses')
5 plt.ylabel('Vendas')
6 plt.show()
```

Argumento
keyword
(kwarg)



Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Como instalar módulos novos

Como temos diversos pacotes existentes, pode ser que durante uma pesquisa, um pacote que você achou interessante não está instalado no seu computador.

Como saber que a biblioteca não está instalada.

Vamos pegar o exemplo ao lado.

Ao tentar importar a biblioteca keyboard, recebi este erro **ModuleNotFoundError**.

Se você digitou corretamente o nome da biblioteca, isso significa que não temos isso instalado.

Para resolver esse problema, vamos usar um instalador que já existe “embutido” dentro do Python, o **PIP**.

```
1 import keyboard

-----
ModuleNotFoundError                                Traceback (most recent call last)
<ipython-input-1-f7cee866d5f5> in <module>
----> 1 import keyboard

ModuleNotFoundError: No module named 'keyboard'
```

Argumento
keyword
(kwarg)

Módulo 17 – Módulos, Bibliotecas e Introdução à Orientação a Objeto – Como instalar módulos novos

Usando o **PIP**, é possível, instalar, desinstalar pacotes.

Caso você queira saber se um pacote está instalado, basta usar o comando abaixo em uma das células do Jupyter:

pip freeze

Caso não encontre na lista o pacote desejado, use o comando abaixo para instalar:

pip install nome da biblioteca

pip freeze

Comando pip freeze

```
argh==0.26.2
asn1crypto==1.3.0
astroid==2.4.2
astropy==4.0.1.post1
atomicwrites==1.4.0
attrs==19.3.0
autopep8 @ file:///tmp/build/80754af9/autopep8_1592412889138/work
Babel==2.8.0
backcall==0.2.0
backports.functools-lru-cache==1.6.1
scipy @ file:///C:/ci/scipy_1592916963468/work
seaborn==0.10.1
selenium==3.141.0
Send2Trash==1.5.0
simplegeneric==0.8.1
singledispatch==3.4.0.3
sip==4.19.13
six==1.15.0
snowballstemmer==2.0.0
sortedcollections==1.2.1
sortedcontainers==2.2.2
soupsieve==2.0.1
Sphinx @ file:///tmp/build/80754af9/sphinx_1594223420021/work
```

Lista de pacotes instalados no
Python

